

単体テストが実装より早い = 設計が悪い (密結合)
" 遅い " 悪い (疎結合)

cf) 1つのテストケース

1. テストにかける労力は最小限に、価値は最大限にすることが目標
単体テストの目的は、設計を見直す機会を創みこと以上に、持続可能性を高めること
網羅率によるテストの品質評価

するべきではないが、今回の論点はそこではない

・コード網羅率 → テストスイートの品質の悪さを示せるも、良さを示すことはできない (コメントを書けば高くなる)

・分岐網羅率 → 分岐の数を拾うため、ロジック依存となり、リファクタリングに影響しない

網羅率とテスト品質の指標は必ずしも一致しない

⇒ 確認不在のテストやライブラリに対する網羅性は低く、テストスイートの品質は評価できない

テストスイートの質を自動的に評価する仕組みはない

品質が悪いのは評価できる

・テストが開発サイクルに含まれる

・ドメインモデル (ビジネスロジックを含む部分) を中心にテストを行う

・最小限の保守コストで、最大限の価値を生み出す → 価値を評価できないといえない

隔離性という単体テストの特性の定義により

2. 隔離性という単体テストの特性の定義により、古典学派とロンドン学派に分かれる

値テストは対象外 (状態テスト)

・ロンドン学派... テストダブル (モック) を用いて、テスト対象システムに依存から分離する

⇒ テスト失敗時の問題が明確になりやすい + 循環依存を断ち切れる

・古典学派... 各テストケース間での依存を分離する

⇒ 共有依存やプロセス外依存に強く、実行時間が短くなりやすい

ただし、テストダブルは作成可能な限りこのことはない

古典学派の方がロンドン学派よりも優れていると考えられる理由

・粒度が適切で、ストーリーボードから分かる (ロンドン学派において、単体キックの振る舞い)

・複雑な依存関係を取り除く郵機が通る (" だと、有無を言わずにモック)

・依存関係を正しく把握するきっかけになる (" によるリットは重要ではない)

・実装の詳細を深くまで把握する必要がない

ロンドン学派における統合テストは、古典学派の単体テストに対応する

Given-When-Then を使っている

3. AAAパターン (3Aパターン)... 単体テストの構造で、準備・実行・確認の3つに分かれる

・1つのテストスイートに同じフェーズを複数用意してはならない (その場合、分割が必要)

・テストケースのコード内に条件分岐を持ち込まない

各フェーズにおけるコードの太玉さは、

・準備フェーズ... 太玉くなく、2も問題ないが、なるべくおろくに上位スコープで定義にしよう

・実行フェーズ... 基本的には、1行で書ける (書けない場合は、カプセル化が必要でないかも)

・確認フェーズ... 1単位の振る舞いを網羅的にテック (9割は9割)

Tips ・テスト対象システムに固定の名前を付ける (sut: system under test など)

・テストスイート内のコード量も膨大ではないよう、空白行とフェーズを表現することによって

テストファイル... テストを実行する際、使用されるオブジェクト (各テスト前に実行される定義共有オブジェクト)

⇒ テストケース間の結びつきを強め、読みやすさを損うので、汎用的にフックメソッドを定義すると良い

テストメソッド名の付け方

なるべく短く、希望的表現 (should など) は避ける

・ビジネスロジック有... 命名規則を特に設けず、非開発者にも通じるような命名 (シリアライズ)

・ " 無... テスト名にテスト対象メソッドの名前を入力してもよい

パラメータ化テスト... 特定のロジックを複数のパラメータでテストする場合、新しくテストメソッドとして分離し、

パラメータを渡してテストすること (正常系と異常系で分けるのもアリ)

実装の詳細は目玉向けに、最終結果に付いてテストするべき
アルゴリズム(=種別付いたテストは、リファクタリング性がある
(=リファクタリングテスト)

4. 良い単体テストを構成する4本の柱

- ・ 退化に対する保護 (重要な部分必細羅的にテストされるべき)
 - ・ リファクタリングへの耐性 (プログラム・ジョンコードをリファクタしても、偽陽性が起こらない)
 - ・ 迅速なフィードバック
 - ・ 保守のしやすさ
- バランス良く含むことが重要

スピン・手書き(自身で実装する)モックという意味合い

5. テスト・ダブル { モック ... 外部への出力を模倣する仕組み (APIリクエストの迷信など) スタブ ... 内部への入力に模倣する仕組み (データのフェイクなど)

コマンド・リクエスト分離の原則 (全てのメソッドは、どちらかに分類されるべき)

- ・ コマンド ... 副作用が刻、値を返さない ⇒ モック
- ・ リクエスト ... " 返す、" 返す ⇒ スタブ

「尋ねるな、命じろ」
= どのような操作をしようか、どのような状態を
もたせようかという情報をお互に公開する

実装の詳細 ... 目標達成のために必要な操作 or 状態 に該当しないもの
⇒ 必要な操作 or 状態、管理しにくいアーキテクチャ (カプセル化)

不変条件は特に

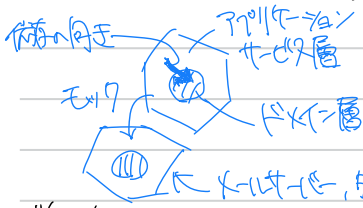
ハキサゴナルアーキテクチャ ... アプリケーション・サービス層とドメイン層を明確に分ける設計

⇒ アプリケーション・サービス層同士の間接は「観測可能な2」

モックを作成して、テスト対象とする

2つないものは原則としてテストしない

→ 1つのアプリケーションの境界を超えるとまじりに使う



6. 単体テストの3つの手法

- ・ 出力値ベーステスト ... 戻り値をテストする手法 副作用のないユースケース、関数型プログラミングに有効
- ・ 状態ベーステスト ... テスト対象システムの状態の変化をテストする手法。プロセス外依存を含む
- ・ コミュニケーションベーステスト ... 協力者オブジェクトとのコミュニケーションをテストする手法 (NLP・XVVDが適用可能)

関数型アーキテクチャ ... 関数的核(不変核)と可変核を明確に分ける設計



低級関数の定義

外部プロセスなどには

2つだけしかなく ⇒ 2つはあたかもブラックボックスのように振る舞う

外部プロセスへの依存は適切にインターフェースを設置した上で、モックとスタブで行う

7. テストの観点で、プログラム・ジョンコードは、以下の2軸で分類される

- ・ 複雑さ、ドメインにおける重要性
- ・ 協力者オブジェクトの数

テストを行うべき部分だけ抽象化する (テスト対象は副作用がない)
ハキサゴナルアーキテクチャにおけるアプリケーション・サービス層

⇒ 過度に複雑なコードと簡単なオブジェクトという設計パターンに基づいてリファクタすると良い

→ ドメインモデルに、プロセス外依存とその協力者オブジェクトを持たせるべきではない (テストが簡単になる)

⇒ 保守の難し、過度に複雑なコードとコントローラとドメイン・アルゴリズムに分けることができる

ドメインにおける重要性が高、協力者オブジェクトの少ないコードは格好のテスト対象

テスト対象を適切に
区別すること

設計が複雑にはなるとトレードオフを考慮するべきになる

⇒ 重要な性質は、

- ・ ドメインモデルのテストのしやすさ
- ・ コントローラの簡潔さ
- ・ パフォーマンスの高さ

コントローラの簡潔さに関しては、
ある程度は妥協が必要になる
⇒ 過度に複雑なコードは生じる

ドメインイベント ... プロセス外依存を専用のイベントで定義することによって抽象化する (インスタンス数?)

⇒ ドメインモデルに紐づけることの2つは責務を拡大させることができる

確認後実行パターン ... Create, Execute, Destroy を利用するパターン

単体テスト対象

具体的には、コントローラのテスト



8. 統合テスト...単体テストの満たす3つの条件の内、1つも満たさずテスト全般

単体テストと同様: Arrange, Act, Assert
1つのユニットテストと単体テストでは対応している異常ケースを全網羅検証するのが良い
単体テストと被らないようにする (確認後実行 (リターンなど))

プロセス外依存
・管理下にある依存...プログラムのDBアクセスなど → テストではそのまま使う
・" (ない) " ... ユーザーサービスや公開API など → " モックで置き換える
→ 中間インターフェースを導入することで、モックの作成のしやすさの観点からGOOD

統合テストの基本指針
・ドメインモデルの境界を明確にする
・アプリケーションを構成する層を減らす (インフラ、サービス、ドメイン) 事足りることが多い
・循環依存を取り除く (依存の方向を定める)

ロジックのテストについては、原則として
・結果可能な振る舞い → するべき (サポートログなど)
・実装の詳細 → しない (診断ログなど)

→ 構造化ログの概念が有用 (ただし、結果可能な振る舞いに属するものはいびり分??)

→ ユニテストテンプレートとプラクティスを用いて、ログの出力方法や出力先をコントロールできるようにする
専用のクラスを作成して、ドメインロジックに追加して、コントローラ内に直接書く必要はない

9. モックに置き換える必要なのは、アプリケーションとプロセス外の境界にあるインターフェース (≒ コントローラ内のもので、直接モックしてはいけない?)
→ 実装の詳細を隠蔽 (メソッド名、関数名など) テストをすることが出来る

これらのケースの場合、モックよりもスピア (書き込みのモック) の方が優れているケースが多い

ロジックシステム (テストケースも含む) (このテストケースは、このテストケースのテスト)

モック利用のベストプラクティス

- ・モックの利用は統合テストに限定する (プロセス外依存は統合テストの対象であるコントローラが持つため)
- ・モックの呼び出し回数と常に確認する
- ・モックの対象になる型は、自身のプロジェクトが所有する型に限定する

= ライブラリをそのままモックにせず、作成したアダプタ (仲介インターフェース) に対して、モックを作成する
ただし、同じプロジェクト内にあるライブラリについては適用外 (例として、Android → 適用外、iOS → 適用内)

10. データベースのテストに必要な事前準備

- ・スキーマをバージョン管理システムを用いて管理する (真実を示す情報源をつくる)
- ・開発者ごとに個別のデータベースインスタンスを用意する (テストの可用性を保護する)
- ・データベースに対する変更は、移行ベースを用いる (どのような変更を加えるのかを開発者が定義する)

移行ベース (スキーマ) 比較ベース (スキーマ) 移行ベース (スキーマ) 比較ベース (スキーマ)

データベース (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ) (スキーマ)

データベース操作とトランザクションは分離すべき

- ・リポジトリ...データベースの操作を行う (リソース)
- ・トランザクション...データベースの操作を反映する (リソース) → 原子性を担保

→ この拡張的な概念として、単位作業 (ビジネスオペレーション) とはシステム側でカラムがある
⇒ 単体のデータの編集を軽減させる

統合テストにおいて、トランザクションのインスタンスを使い回すことは厳禁

テストデータのライフサイクル
・テストケースを1つずつ走らせる (並列に走らせる)

・テストに残ったデータの復元を要する (基底リソースとして定義するのが重要)

統合テストのコード量は増えやすいので、適度にトリックやヘルプトリックを作成するとよい

cf.) オブジェクトマッパー... テストを実行する際に必要なテストリソースの生成を助けるリソースやトリック

11. プライベートなメソッドは、実装の詳細に該当することが多い → テスト対象にするべきではない
cf.) テストは観測可能な振る舞いに対して行う (プライベートは隠されたものは間接的にテストを行う)
⇒ もしテストすべきものが隠れている場合、抽象化の下落が起きている可能性あり (ドメインロジックなど)
ドメイン知識がテストコードに漏れ出しはならない (テスト対象の実装の詳細と重複してはいけな)
⇒ テストデータと期待値を直接テストデータとして用意しておくべき (インライン形式が扱いやすい)
対象メソッドにテストダブルを使用すべき場合、単一責任の原則を破っている可能性あり → テストダブルはインロジック
⇒ 適切にリファクタリングを行い、分離する必要がある
現在日時を取り扱うについては、テストの整合性を担保するために、値として準備するのが安全

